

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «МУЗЕЙ»

Студента 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Заблоцького Івана Юрійовича

Керівник: к.т.н., доц. Шестопапов С.В.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.

(підпис) (прізвище та ініціали)

_____ Шестопапов С.В.

(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

ЗМІСТ

.....	стор.
ВСТУП	3
ЗАВДАННЯ	4
1. ТЕОРЕТИЧНА ЧАСТИНА	5
1.1 Поняття ООП	5
1.2 Опис об'єктної частини	6
2. ПРАКТИЧНА ЧАСТИНА	7
2.1 Опис класі.....	7
2.2 Інтерфейс.....	10
ВИСНОВОК.....	13
СПИСОК ЛІТЕРАТУРИ.....	14

ВСТУП

Метою даної курсової роботи є розробка програми мовою C# на тему «Музей» із застосуванням основних принципів об'єктно-орієнтованого програмування: інкапсуляції, успадкування та поліморфізму. Розроблена система моделює структуру музею, що складається з кімнат (залів) та різноманітних експонатів.

Відповідно до варіанта завдання необхідно реалізувати ієрархію класів, яка включає базовий клас «Експонат» та його нащадків: «Картина», «Об'ємний експонат», «Скульптура», «Археологічний експонат» та «Науковий прилад». Окремо реалізується клас «Кімната», що описує параметри залу музею та забезпечує перевірку можливості розміщення в ньому конкретного експоната.

Усі поля класів є закритими, а доступ до них здійснюється через властивості з відповідною валідацією вхідних даних. Взаємодія з програмою реалізована через консольний інтерфейс, що дозволяє переглядати наявні експонати, додавати нові до певного залу, а також додавати нові кімнати.

У ході виконання роботи закріплюються практичні навички проєктування ієрархій класів, роботи з колекціями, обробки виняткових ситуацій та побудови консольних застосунків мовою C#.

ЗАВДАННЯ

Варіант 8б. Створення ієрархії класів на тему «Музей».

Створити клас кімната, що задає розміри кімнати: ширина, довжина, висота, корисна площа стін (перевірити, щоб корисна площа стін була не більша від загальної площі). Створити клас «Експонат» (автор, країна, рік, врахувати, що автор може бути невідомий), зі спадкоємцями «Картина» (розміри: ширина, висота), «Об'ємний експонат» (ширина, довжина, висота, необхідна для розміщення експонату), у об'ємного експонату спадкоємці «Скульптура», прилад».

Реалізувати перевірку, чи можна помістити експонат у кімнату.

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтоване програмування (ООП) – це парадигма програмування, що базується на поданні програми у вигляді сукупності взаємодіючих об'єктів, кожен з яких є екземпляром певного класу[1]. Класи описують властивості та поведінку об'єктів, а між класами може існувати ієрархія наслідування. ООП дозволяє створювати програми, структура яких відображає реальний світ, що значно спрощує їх розробку та подальше супроводження.

Кожний стиль програмування має свою концептуальну базу. Для ООП стилю такою базою є об'єктна модель. Її побудова основана на використанні 3-х основних концепцій[2,3]:

- а) інкапсуляція;
- б) поліморфізм;
- с) наслідування.

Інкапсуляція – це механізм, який поєднує дані та методи, що працюють з цими даними, в єдину сутність – об'єкт, та обмежує прямий доступ до внутрішніх деталей його реалізації. Завдяки інкапсуляції стан об'єкта може змінюватися лише через визначений інтерфейс (властивості та методи), що захищає дані від некоректного використання. У даній роботі інкапсуляція реалізована через оголошення полів класів як `private` з наданням доступу до них виключно через публічні властивості з валідацією значень[1].

Наслідування – це механізм, що дозволяє одному класу (нащадку) отримувати властивості та методи іншого класу (батьківського). Нащадок може розширювати або змінювати поведінку батьківського класу, додаючи власні поля та методи. У даній роботі клас `VolumeExhibit` наслідує клас `Exhibit`, а класи `Sculpture`, `ArchaeologicalExhibit` та `ScientificInstrument` наслідують `VolumeExhibit`, утворюючи дворівневу ієрархію[1].

Поліморфізм – це властивість, яка дозволяє об'єктам різних класів відповідати на одне й те саме звернення по-різному, виходячи зі свого типу. На практиці поліморфізм реалізується через перевизначення методів у класах-нащадках. У даній роботі кожен клас-нащадок перевизначає метод `ToString()`, повертаючи рядок з описом, характерним саме для свого типу експоната[1].

1.2 Опис об'єктної частини

Програма реалізує консольний застосунок для управління колекцією музейних експонатів. Система підтримує чотири категорії експонатів: картини, скульптури, археологічні знахідки та наукові інструменти. Кожен тип експонатів має власний набір характеристик, але всі вони успадковують спільні властивості від базового класу `Exhibit`.

Музей складається з кімнат (залів), які характеризуються шириною, довжиною, висотою та корисною площею стін. Кожна кімната містить список своїх експонатів. Об'ємні експонати (скульптури, археологічні знахідки, наукові інструменти) можуть бути розміщені в залі лише в тому випадку, якщо їхній об'єм та габарити не перевищують доступний простір кімнати.

Користувач взаємодіє із системою через консольне меню, де може переглядати наявні експонати, додавати нові до певного залу, а також додавати нові кімнати до музею. При введенні некоректних даних програма виводить відповідне повідомлення про помилку та пропонує спробувати ще раз.

Діаграму класів наведено в додатку А. Розглянемо більш детально структуру кожного класу.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Діаграму класів наведено в додатку А. Розглянемо більш детально структуру кожного класу.

- 1) Базовий клас «Exhibit» – базовий клас експонату. Зберігає загальні дані: назва, автор, країна, рік. Валідує поля при встановленні. Метод ShowYear() форматує рік у форматі «до н.е. / н.е.».
 - a) Приватні поля: name, author, country, year.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Властивості: Name, Author, Country, Year.
 - ShowYear(int currentYear) – повертає рядок із роком у форматі «р. н.е.» або «р. до н.е.».
- 2) Клас «VolumeExhibit», нащадок класу «Exhibit» – об'ємний експонат. Розширює Exhibit трьома вимірами: ширина, довжина, висота (всі double, невід'ємні). Метод FitsInRoom() перевіряє, чи вміщується експонат у кімнату за габаритами.
 - a) Приватні поля: width, length, height.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Властивості: Width, Length, Height (не можуть бути від'ємними).
 - Перевизначення ToString – повертає рядок з описом об'ємного експоната.

- FitsInRoom(Room room) – перевіряє, чи вміщається експонат за габаритами у задану кімнату.
- 3) Клас «Picture», нащадок класу «Exhibit» – картина – Плaskий експонат з висотою та шириною (int). Метод FitsInRoom() перевіряє відповідність розмірів стіни кімнати.
- a) Приватні поля: height, width.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Властивості: Height, Width (не можуть бути від'ємними).
 - Перевизначення ToString – повертає опис картини.
 - FitsInRoom(Room room) – перевіряє, чи підходить розмір картини для розміщення на стіні кімнати.
- 4) Клас «Sculpture», нащадок класу «VolumeExhibit» - скульптура. Дочірній клас VolumeExhibit, параметри розміру – int. Лише перевизначає ToString().
- a) Приватні поля: успадковані від VolumeExhibit: width, length, height.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Перевизначення ToString – повертає рядок з описом скульптури.
- 5) Клас «ArchaeologicalExhibit», нащадок класу «VolumeExhibit» – археологічний експонат. Аналогічний до Sculpture, але семантично позначає артефакти розкопок. Лише перевизначає ToString().
- a) Приватні поля: успадковані від VolumeExhibit: width, length, height.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами

- с) Методи:
 - Перевизначення ToString – повертає рядок з описом археологічного експоната з підтримкою від'ємних років (до н.е.).
- 6) Клас «ScientificInstrument», нащадок класу «VolumeExhibit» – науковий інструмент. Те саме, що попередні нащадки VolumeExhibit, але розміри – double. Лише перевизначає ToString().
 - а) Приватні поля: успадковані від VolumeExhibit: width, length, height (тип double для точних вимірів).
 - б) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - с) Методи:
 - Перевизначення ToString – повертає рядок з описом наукового інструменту.
- 7) Клас «Room» – кімната музею. Має розміри (ширина, довжина, висота), корисну площу стін та список експонатів List<Exhibit>.
 - а) Приватні поля: width, length, height, usefulWallArea, volume.
 - б) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - с) Лісти:
 - ExhibitList – публічна властивість типу List<Exhibit> для збереження переліку експонатів кімнати.
 - д) Методи:
 - Властивості: Width, Length, Height, UsefulWallArea.
 - CanAccommodate(double L, double W, double H) – перевіряє наявність вільного об'єму та зменшує його після успішного розміщення.
 - Перевизначення ToString – повертає інформацію про кімнату.

2.2 Інтерфейс

Готовий проєкт розміщено в [4]. Під час запуску програми користувач бачить вступне вікно (Рис. 2.1)

```
ви прийшли до музею, щоб подивитися на експонати:
1 передивитись експонати
2 додати експонат
3 додати кімнату
4 вийти
```

Рисунок 2.1 – початкове вікно

Користувач потрапляє сюди коли запускає програму, або виходить з музею для вибору іншого параметру (Рис.2.1).

Якщо користувач натискає «1», він переходить до перегляду експонатів які вже є в музеї (Рис. 2.2)

```
Картина: назва = Мона Ліза, автор = Леонардо да Вінчі, країна = Італія, рік = 1503 р. н.е., висота = 53, ширина = 77
Археологічний експонат: назва = Тиранозавр Рекс, автор = Тиранозавр, країна = США, рік = 65000000 р. до н.е., ширина = 1
2, довжина = 2, висота = 4
Скульптура: назва = Давид, автор = Мікеланджело, країна = Італія, рік = 1504 р. н.е., ширина = 5, довжина = 2, висота =
1
Науковий інструмент: назва = Телескоп Ганса Янссена, автор = Ганс Янссен, країна = Нідерланди, рік = 1608 р. н.е., ширин
а = 0,5, довжина = 0,15, висота = 0,2
```

Рисунок 2.2 – наявні експонати

У випадку натискання «2», користувачу буде надана можливість додати експонат (Рис. 2.3) для успішного додання експоната користувачу потрібно ввести всі параметри які запрошує програма.

```

Введіть тип експонату (1-картина, 2-скульптура, 3-археологічний експонат, 4-науковий інструмент):
1
Введіть автора експонату:
Да Вінчі
Введіть країну експонату:
Італія
Введіть рік створення експонату:
1454
Введіть висоту картини:
122
Введіть ширину картини:
66
Введіть назву картини:
Мона Ліза

```

Рисунок 2.3 – додавання експонату

Якщо введений експонат більше ніж кімната видає помилку (Рис. 2.4)

```

Помилка: Експонат 'ggg' не поміщається в кімнату.
|

```

Рисунок 2.4 – помилка введення експонату

При натисканні «3», користувач зможе додати нову кімнату ввівши її розміри (Рис. 2.5)

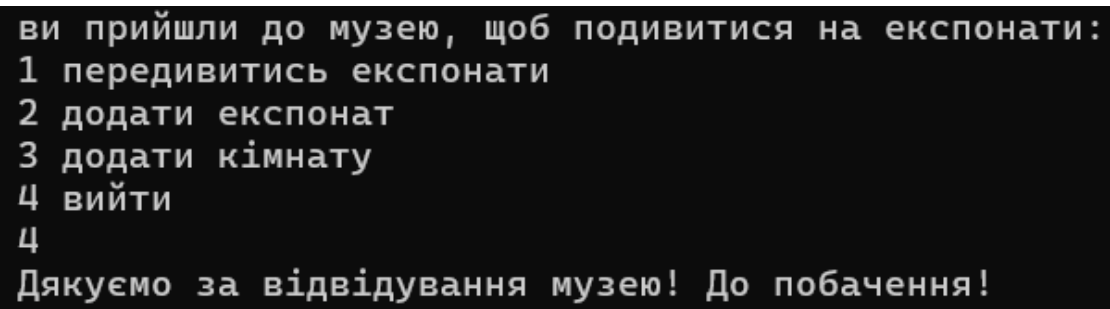
```

Введіть ширину кімнати (ціле число):
15
Введіть довжину кімнати (ціле число):
25
Введіть висоту кімнати (ціле число):
10
Введіть корисну площу стін (ціле число):
100

```

Рисунок 2.5 – додавання кімнати

У разі натискання «4», користувач виходить з музею (Рис 2.6)



```
ви прийшли до музею, щоб подивитися на експонати:  
1 передивитись експонати  
2 додати експонат  
3 додати кімнату  
4 вийти  
4  
Дякуємо за відвідування музею! До побачення!
```

Рисунок 2.6 – вихід з музею

ВИСНОВОК

У ході виконання курсової роботи було розроблено програму на мові С# з використанням принципів об'єктно-орієнтованого програмування на тему «Музей». У процесі роботи було створено ієрархію класів для представлення експонатів та кімнат, а також реалізовано взаємодію між ними.

Під час розробки програми були використані основні принципи ООП: інкапсуляція, наслідування та поліморфізм. Для кожного класу були створені відповідні поля, властивості, конструктори та методи.

У результаті виконання курсової роботи були закріплені практичні навички програмування на С#, роботи з класами, колекціями, методами, властивостями та консольним інтерфейсом. Розроблена програма демонструє можливість створення повноцінної системи управління музеєм засобами об'єктно-орієнтованого програмування.

СПИСОК ЛІТЕРАТУРИ

1. Microsoft Documentation. Object-Oriented Programming (C#). [Електронний ресурс]. – Режим доступу: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop>
2. W3Schools. C# OOP (Object-Oriented Programming). [Електронний ресурс]. – Режим доступу: https://www.w3schools.com/cs/cs_oop.php
3. Віктор Є.О., Геренко О.А., Шпінарева І.М. Практикум з курсу «Об'єктно-орієнтоване програмування мовою C#». [Електронний ресурс] / Одеса, 2026. – Режим доступу: <https://oop-practicum-csharp-851f60.gitlab.io/>
4. Репозиторій GitHub – вихідний код курсової роботи. [Електронний ресурс]. – Режим доступу: <https://github.com/vanyazablotski012-sketch/kursach>

Додаток А. Діаграма класів

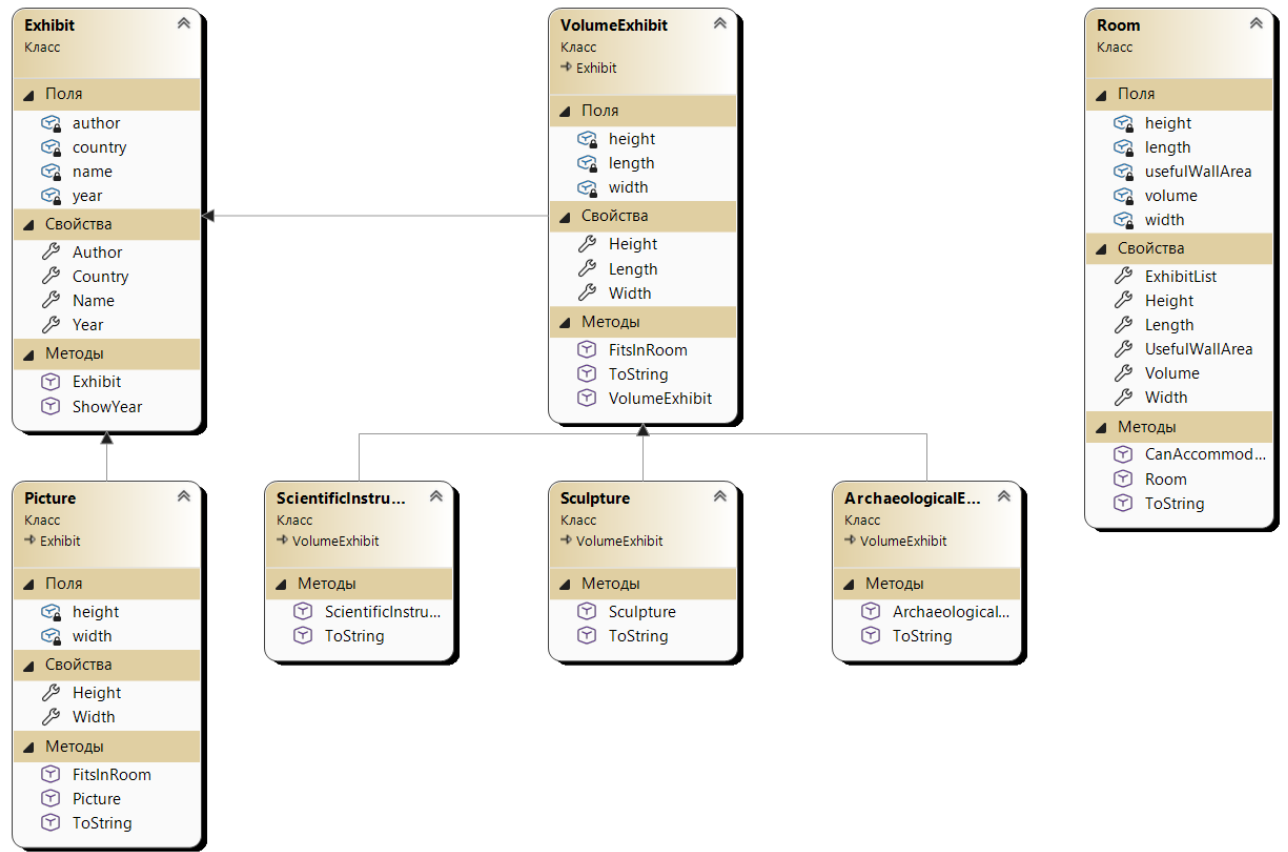


Рисунок А1. – Діаграма класів

Додаток Б. Код класу

```

public class Exhibit
{
    private string name;
    private string author;
    private string country;
    private int year;

    public Exhibit(string name, string author, string country, int year)
    {
        Name = name;
        Author = author;
        Country = country;
        Year = year;
    }

    public string Name
    {
        get { return name; }
        set
        {
            if (string.IsNullOrEmpty(value)) throw new ArgumentException("Назва
не може бути пустою.");
            name = value;
        }
    }

    public string Author
    {
        get { return author; }
        set
        {
            if (string.IsNullOrEmpty(value)) author = "Невідомий";
            author = value;
        }
    }

    public string Country
    {
        get { return country; }
        set
        {
            if (string.IsNullOrEmpty(value)) throw new
ArgumentException("Країна не може бути пустою.");
            country = value;
        }
    }

    public int Year
    {
        get { return year; }
        set
        {
            if (value != 0)
                year = value;
            else
                throw new ArgumentException("Рік не може бути нулем.");
        }
    }

    public string ShowYear(int currentYear)
    {
        if (currentYear > 0)
            return $"{currentYear} р. н.е.";
        else
            return $"{-currentYear} р. до н.е.";
    }
}

```

Лістинг Б.1 – Клас Exhibit


```

public class Room
{
    private int width;
    private int length;
    private int height;
    private int usefulWallArea;
    private double volume;
    private List<Exhibit> exhibitList = new List<Exhibit>();

    public Room(int width, int length, int height, int usefulWallArea)
    {
        Width = width;
        Length = length;
        Height = height;
        UsefulWallArea = usefulWallArea;
        volume = width * length * height;
    }

    public int Width
    {
        get { return width; }
        set
        {
            if (value < 0) throw new ArgumentException("Ширина не може бути
від'ємною.");
            width = value;
        }
    }
    public int Length
    {
        get { return length; }
        set
        {
            if (value < 0) throw new ArgumentException("Довжина не може бути
від'ємною.");
            length = value;
        }
    }
    public int Height
    {
        get { return height; }
        set
        {
            if (value < 0) throw new ArgumentException("Висота не може бути
від'ємною.");
            height = value;
        }
    }
    public int UsefulWallArea
    {
        get { return usefulWallArea; }
        set
        {
            if (value > Width * Height * 2 + Length * Height * 2) throw new
ArgumentException("Корисна площа стін не може перевищувати загальну площу стін.");
            usefulWallArea = value;
        }
    }
    public List<Exhibit> ExhibitList
    {
        get{ return exhibitList; }
    }
    public double GetVolume()
    {
        return (double)Width * Length * Height;
    }
    public void AddExhibit(Exhibit exhibit)

```

```

{
    if (exhibit is VolumeExhibit ve)
    {
        if (!CanAccommodate(ve.Length, ve.Width, ve.Height))
            throw new InvalidOperationException($"Експонат '{exhibit.Name}' не
поміщається в кімнату.");
    }
    ExhibitList.Add(exhibit);
}
public bool CanAccommodate(double L, double W, double H)
{
    if (volume >= L * W * H
        && (Width >= W) && (Length >= L) && (Height >= H))
    {
        volume -= L * W * H;
        return true;
    }
    return false;
}

public override string ToString()
{
    return $"Кімната: ширина = {Width}, довжина = {Length}, висота = {Height},
корисна площа стін = {UsefulWallArea}";
}
}

```

Лістинг Б.2 – Клас Room, аркуш 2

```

namespace kursach.classes
{
    public class ArchaeologicalExhibit : VolumeExhibit
    {
        public ArchaeologicalExhibit(string name, string author, string country, int
year, int width, int length, int height) : base(name, author, country, year, width,
length, height)
        {
        }
        public override string ToString()
        {
            return $"Археологічний експонат: назва = {Name}, автор = {Author}, країна =
{Country}, рік = {ShowYear(Year)}, ширина = {Width}, довжина = {Length}, висота =
{Height}";
        }
    }
}

```

Лістинг Б.3 – Клас ArchaeologicalExhibit

```

namespace kursach.classes
{
    public class Picture : Exhibit
    {
        private int height;
        private int width;
        public Picture(string name, string author, string country, int year, int height,
int width) : base(name, author, country, year)
        {
            Height = height;
            Width = width;
        }
        public int Height

```

Лістинг Б.4 – Клас Picture, аркуш 1

```

{
    get { return height; }
    set {
        if (value < 0) {
            throw new ArgumentException("Висота не може бути від'ємною.");
        }
        height = value;
    }
}
public int Width
{
    get { return width; }
    set {
        if (value < 0) {
            throw new ArgumentException("Ширина не може бути від'ємною.");
        }
        width = value;
    }
}
public override string ToString()
{
    return $"Картина: назва = {Name}, автор = {Author}, країна = {Country}, рік
= {ShowYear(Year)}, висота = {Height}, ширина = {Width}";
}
public bool FitsInRoom(Room room)
{
    return Width <= room.Width &&
        Height <= room.Height;
}
}
}

```

Лістинг Б.4 – Клас Picture, аркуш 2

```

namespace kursach.classes
{
    public class VolumeExhibit : Exhibit
    {
        private double width;
        private double length;
        private double height;
        public VolumeExhibit(string name, string author, string country, int year,
double width, double length, double height) : base(name, author, country, year)
        {
            Width = width;
            Length = length;
            Height = height;
        }
        public double Width
        {
            get { return width; }
            set
            {
                if (value < 0) throw new ArgumentException("Ширина не може бути
від'ємною.");
                width = value;
            }
        }
        public double Length
        {
            get { return length; }
            set
            {
                if (value < 0) throw new ArgumentException("Довжина не може бути
від'ємною.");
            }
        }
    }
}

```

Лістинг Б.5 – Клас VolumeExhibit, аркуш 1

```

        length = value;
    }
}
public double Height
{
    get { return height; }
    set
    {
        if (value < 0) throw new ArgumentException("Висота не може бути
від'ємною.");
        height = value;
    }
}
public override string ToString()
{
    return $"Об'ємний експонат: назва = {Name}, автор = {Author}, країна =
{Country}, рік = {ShowYear(Year)}, ширина = {Width}, довжина = {Length}, висота =
{Height}";
}
public bool FitsInRoom(Room room)
{
    return Width <= room.Width &&
        Length <= room.Length &&
        Height <= room.Height;
}
}
}

```

Лістинг Б.5 – Клас VolumeExhibit, аркуш 2

```

namespace kursach.classes
{
    public class Sculpture : VolumeExhibit
    {
        public Sculpture(string name, string author, string country, int year, int
width, int length, int height) : base(name, author, country, year, width, length,
height)
        {
        }
        public override string ToString()
        {
            return $"Скульптура: назва = {Name}, автор = {Author}, країна = {Country},
рік = {ShowYear(Year)}, ширина = {Width}, довжина = {Length}, висота = {Height}";
        }
    }
}

```

Лістинг Б.6 – Клас Sculpture

```

namespace kursach.classes
{
    public class ScientificInstrument : VolumeExhibit
    {
        public ScientificInstrument(string name, string author, string country, int
year, double width, double length, double height) : base(name, author, country, year,
width, length, height)
        {
        }
        public override string ToString()
        {
            return $"Науковий інструмент: назва = {Name}, автор = {Author}, країна =
{Country}, рік = {ShowYear(Year)}, ширина = {Width}, довжина = {Length}, висота =
{Height}";
        }
    }
}

```

Лістинг Б.7 – Клас ScientificInstrument